SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

OOPM

Module 3

- Inheritance
- Types of Inheritance
- Polymorphism
- Method overloading
- Method Overriding
- Final class and Method
- Abstract class
- Interface

What is Inheritance

- The mechanism of deriving a new class from an old one is called inheritance
- The old class is known as the base class or super class
- The new class is known as the subclass or derived class or child class

Types of inheritance

- Single inheritance(only super class)
- Heirarchical inheritance(one super class,many subclasses)
- Multilevel inheritance(derived from a derived class)
- Multiple inheritance(many superclasses) implemented in the form of interface

Syntax to inherit a class

```
class subclass-name extends superclass-name
{
//body of class
}
```

Example

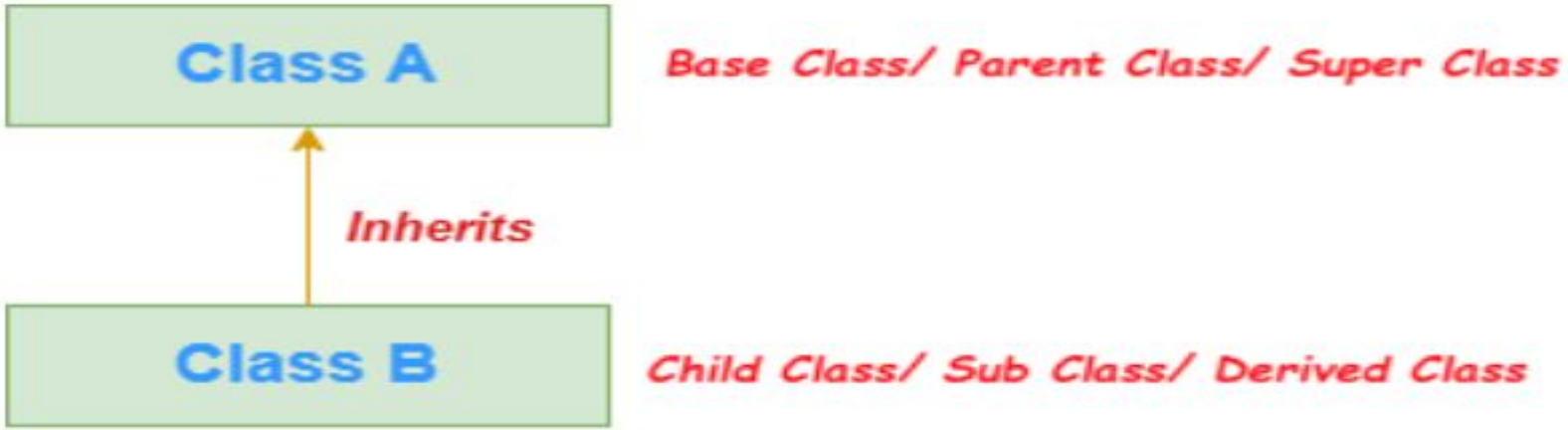
```
class Parent {
    void show() {
        System.out.println("This is the parent class.");
    }
}

class Child extends Parent {
    void display() {
        System.out.println("This is the child class.");
    }
}

public class Main {
    public static void main(String[] args) {
        Child c = new Child();
        c.show();      // inherited from Parent
        c.display();   // own method
    }
}
```

Single Inheritance

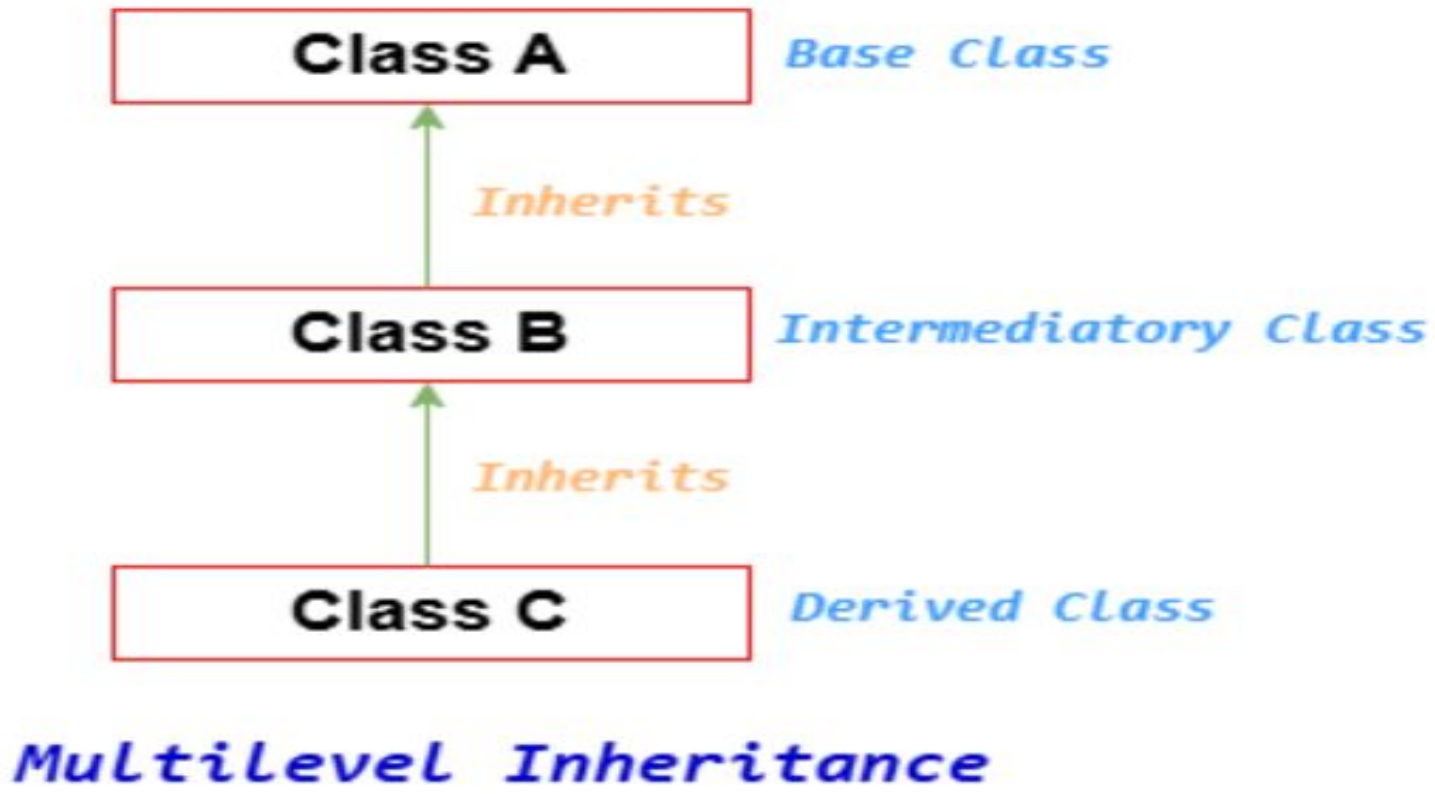
When a class inherits another class, it is known as a single inheritance.



Single Inheritance

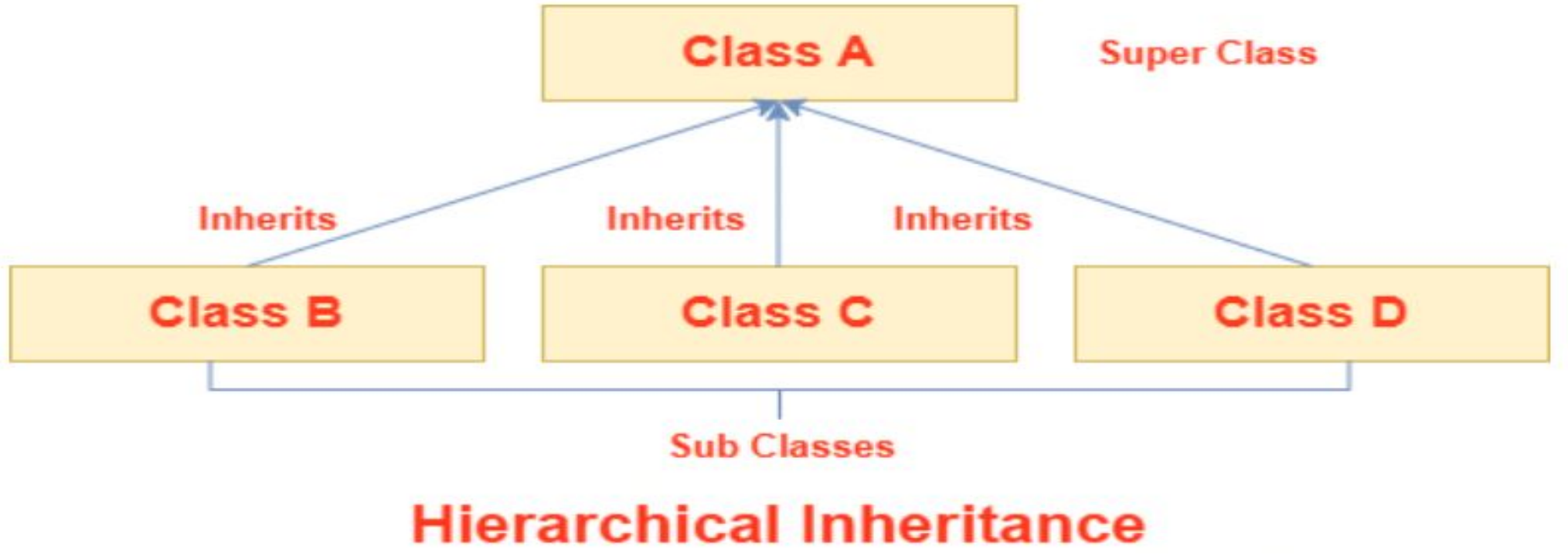
Multilevel Inheritance

When there is a chain of inheritance, it is known as *multilevel inheritance*.



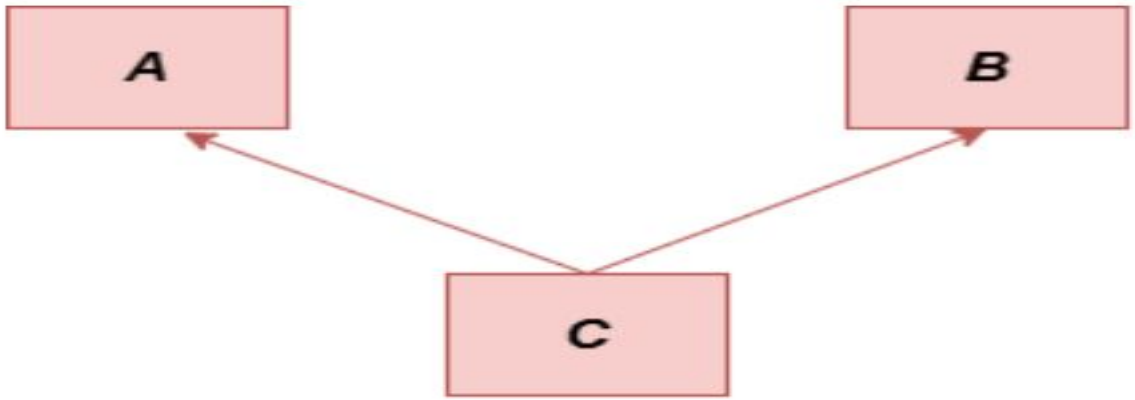
Hierarchical Inheritance

When two or more classes inherits a single class, it is known as hierarchical inheritance.



Multiple Inheritance

A class's capacity to inherit traits from several classes is referred to as multiple inheritances. This notion may be quite helpful when a class needs features from many sources.



Multiple Inheritance

Multiple inheritances, however, can result in issues like the diamond problem, which occurs when two superclasses share the same method or field and causes conflicts.
Suppose there are three classes A, B, and C. The C class inherits A and B classes. If A and B classes have the same method and we call it from child class object, there will be ambiguity to call the method of A or B class. It is called diamond problem.
Java uses interfaces to implement multiple inheritances in order to prevent these conflicts.

Single Inheritance

```
class Vehicle {
    void startEngine() {
        System.out.println("Engine started.");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving.");
    }
}

public class Main {
    public static void main(String[] args) {
        Car car = new Car();
        car.startEngine(); // Inherited from Vehicle
        car.drive();       // Car's own method
    }
}
```

Multilevel Inheritance


```
class Vehicle {
    void startEngine() {
        System.out.println("Engine started.");
    }
}
class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving.");
    }
}
class ElectricCar extends Car {
    void chargeBattery() {
        System.out.println("Battery is charging.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        ElectricCar eCar = new ElectricCar();
        eCar.startEngine();          // from Vehicle
        eCar.drive();                // from Car
        eCar.chargeBattery();        // from ElectricCar
    }
}
```

Hierarchical Inheritance


```
class Vehicle {
    void startEngine() {
        System.out.println("Engine started.");
    }
}
class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving.");
    }
}
class Truck extends Vehicle {
    void loadCargo() {
        System.out.println("Truck is loading cargo.");
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        Car car = new Car();
        car.startEngine();
        car.drive();

        Truck truck = new Truck();
        truck.startEngine();
        truck.loadCargo();
    }
}
```

Super Keyword

- super is a **reference keyword** in Java used to refer to the **immediate parent class object**.
- It is mainly used in **inheritance**.

There are **3 primary uses**:

•Call parent class constructor

•Access parent class variable

•Call parent class method

Call parent class constructor

- Used when child wants to initialize parent properties.

```
class Shape {
    String color;

    Shape(String color) {
        this.color = color;
    }
}

class Circle extends Shape {
    double radius;

    Circle(String color, double radius) {
        super(color);   // calls Shape constructor
        this.radius = radius;
    }
}
```

Access Parent Class Variable

Used when parent and child have **same variable name**.

```
class Parent {
    String name = "Parent";
}
class Child extends Parent {
    String name = "Child";
    void display() {
        System.out.println(name);        // Child
        System.out.println(super.name); //
Parent
    }
}
```

```
class Super_param
{
    public static void main(String args[])
    {
     Child c=new Child();
     c.display();
    }
}
```

Call Parent Class Method

Used when child overrides a method but still wants parent logic..

```
class Parent {
    void show() {
        System.out.println("Parent method");
    }
}
class Child extends Parent {
    void show() {
        super.show();   // calls parent method
        System.out.println("Child method");
    }
}
```

```
class Super_method
{
    public static void main(String args[])
    {
      Child c=new Child();
      c.show();
    }
}
```

Constructor Chaining Diagram

Child() constructor

↓

super() → Parent() constructor

↓

Initialize parent data

↓

Return to Child()

↓

Initialize child data

Topics related to polymorphism

- Method overriding
- Abstract method
- Interface(multiple inheritance)
- Final keyword

Method overriding

- In a class hierarchy, when a method in a subclass has the **same name and type signature** as a method in its superclass, then the method in the subclass is said to override the method in the superclass.

Abstract class

- Abstract class is a class wherein you only define the methods and the actual implementation is in the derived class
- No objects can be created for the abstract class
- The keyword used is abstract
- The subclass derived from the abstract class must implement all the abstract methods
- Abstract constructors and static methods cannot exist

Final keyword

- It is used to create equivalent of constants
- Methods declared cannot be overriden
- Class declared as final cannot be inherited

Simple program that demonstrates final keyword


```
final class Vehicle {   // Final class cannot be extended
    final int speedLimit = 100;  // Final variable (constant)

    final void displaySpeed() {  // Final method cannot be overridden
        System.out.println("Speed limit is: " + speedLimit + " km/h");
    }
}
// Trying to extend Vehicle will give an error
// class Car extends Vehicle { }   // ❌ Not allowed

class CarExample {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();

        // Accessing final variable
        v.displaySpeed();

        // v.speedLimit = 120;   // ❌ Error: cannot assign a value to final variable
    }
}
```

Method Overloading

- **Method overloading** means having **multiple methods with the same name** in a class but **different parameter lists**.
- The compiler decides which method to call based on the **number** and **type** of arguments.
- It is an example of **compile-time polymorphism**.

Polymorphism

- The word *polymorphism* means **"many forms"**.
- In Java, polymorphism allows the **same method or object** to behave differently in different contexts.
- There are **two types of polymorphism**:
- **Compile-time polymorphism (Method Overloading)**
  - Decided at **compile time**.
  - Achieved by **method overloading**.
- **Runtime polymorphism (Method Overriding)**
  - Decided at **runtime**.
  - Achieved by **method overriding + inheritance + dynamic method dispatch**.

Interface

- An **interface** in Java is like a **blueprint** of a class.
- It contains only **abstract methods**.
- Keyword used is implement
- A class must **implement** an interface to provide the method definitions.
- **Multiple inheritance** is possible in Java using interfaces.

```
// First interface
interface Printer {
    void print(String document);
}

// Second interface
interface Scanner {
    void scan(String document);
}

// Class implements multiple interfaces
class MultiFunctionMachine implements Printer, Scanner {

    @Override
    public void print(String document) {
        System.out.println("Printing: " + document);
    }

    @Override
    public void scan(String document) {
        System.out.println("Scanning: " + document);
    }
}
```

```
// Main class
public class Interface_2{
    public static void main(String[] args) {
        MultiFunctionMachine mfm = new MultiFunctionMachine();

        // Using as Printer
        Printer p = mfm;
        p.print("Report.pdf");

        // Using as Scanner
        Scanner s = mfm;
        s.scan("Invoice.png");
    }
}
```

- Shows **multiple interface implementation**
- (Java allows this unlike multiple inheritance of classes).
- Demonstrates **polymorphism**:
  the same object mfm can be treated as a Printer or a Scanner.

Early vs Late Binding in Java

- Binding = Linking a method call to its method body.

Two types:

- Early Binding (Static Binding)
- Late Binding (Dynamic Binding)

Early Binding (Static Binding)

- Occurs at Compile Time.
- Method call is resolved before execution.

Used in:

- Method Overloading
- Static Methods
- Final Methods
- Private Methods

Example – Early Binding


```
class Test {
  void show(int a) {}
  void show(String s) {}
}

Test t = new Test();
t.show(10);
```

Compiler decides method → Compile Time

Late Binding (Dynamic Binding)

- Occurs at Runtime.
- Method call is resolved during execution.

Used in:

- Method Overriding
- Runtime Polymorphism
- Non-static Methods

Example – Late Binding


```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
public class Main {
    public static void main(String[] args)
    {
        Animal a = new Dog();
        a.sound();   // decided at runtime
    }
}
```

Reference → Animal

Object → Dog

- Method decided at Runtime

Early vs Late Binding – Difference

- Early Binding:
- Compile Time
- Faster
- Overloading

- Late Binding:
- Runtime
- More Flexible
- Overriding

Early vs Late Binding Comparison Table

Feature	Early Binding	Late Binding
Also Called	Static Binding	Dynamic Binding
Decision Time	Compile Time	Runtime
Polymorphism Type	Compile-time	Runtime
Method Type	Overloading, Static, Final, Private	Overriding
Flexibility	Less Flexible	More Flexible

Garbage collector

- The Java runtime environment has a garbage collector that periodically frees the memory used by objects that are no longer needed.
- Two basic approaches used by garbage collectors are
  - Reference counting
  - Tracing.

Reference counting

- It maintains a reference count for every object.
- A newly created object will have count as 1.
- Throughout its lifetime, the object will be referred to by many other object thus incrementing the reference count.
- As the referencing object move to other objects, the reference count for that particular object is decremented.
- When reference count for a particular object is 0, the object can be garbage collected.

Garbage collector

Tracing

- This technique traces the entire set of objects (starting from root) and all objects having reference on them are marked in some way.
- Tracing garbage collector algorithm popularly known as is mark and sweep garbage collector scans Java's dynamic memory areas for objects, marking those objects that are referenced.
- After all the objects are investigated, the objects that are not marked (not referenced) are assumed to be garbage and their memory is reclaimed.
- Mark and sweep collectors further use the techniques of Compaction and Copying for fragmentation problems that may arise once you sweep the unreferenced objects.
- Compaction moves all the live objects towards one end making the other end a large free space.
- Copying techniques copies all live objects besides each other into a new space and the old space is considered free now.

Garbage collector

- The garbage collector runs either synchronously or asynchronously in a low priority daemon thread.
- The garbage collector executes synchronously when the system runs out of memory or asynchronously when the system is idle.
- The garbage collector can be invoked to run at any time by calling System.gc() or Runtime.gc().
- But asking the garbage collector to run does not guarantee that your objects will be garbage collected.